AELL 101
Lecture Notes
Group 12
April 22nd - April 29th

Sumedh Nitin Jamsandekar
Aaron Binoj Amacadu
Abhimanyu Prakash
Naman Rindani
Sarvesh Saravanan

Department of Electrical Engineering
Indian Institute of Technology Delhi-Abu Dhabi

Lecture 1: April 22nd 2025

1 Digital Circuits

1.1 Boolean Logic

Boolean logic deals with two values:

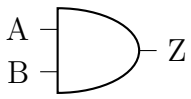
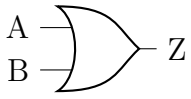
$$\text{True} = 1, \quad \text{False} = 0$$

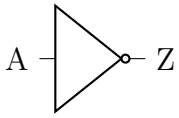
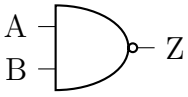
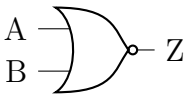
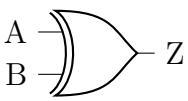
In digital systems:

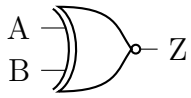
- **High logic (1)**: "on" or high voltage (e.g., 3.3V or 5V).
- **Low logic (0)**: "off" or low voltage (close to 0V).

These two states, high and low logic, are the foundation of all digital circuits and electronics.

2 Basic Logic Gates

Gate Name	Explanation	Operator	Truth Table	Logic Circuit
AND	Output is 1 only when both inputs are 1. Otherwise, output is 0.	$Z = A \cdot B$	A B Z 0 0 0 0 1 0 1 0 0 1 1 1	
OR	Output is 1 if at least one input is 1.	$Z = A + B$	A B Z 0 0 0 0 1 1 1 0 1 1 1 1	

Gate Name	**Explanation**	**Operator**	**Truth Table**	**Logic Circuit**
NOT	Inverts the input.	$Z = \overline{A}$	A | Z 0 | 1 1 | 0	
NAND	Inverse of AND.	$Z = \overline{A \cdot B}$	A | B | Z 0 | 0 | 1 0 | 1 | 1 1 | 0 | 1 1 | 1 | 0	
NOR	Inverse of OR.	$Z = \overline{A + B}$	A | B | Z 0 | 0 | 1 0 | 1 | 0 1 | 0 | 0 1 | 1 | 0	
XOR	Exclusive OR.	$Z = A \oplus B$	A | B | Z 0 | 0 | 0 0 | 1 | 1 1 | 0 | 1 1 | 1 | 0	

Gate Name	Explanation	Operator	Truth Table	Logic Circuit
XNOR	Exclusive NOR.	$Z = \overline{A \oplus B}$	A B Z 0 0 1 0 1 0 1 0 0 1 1 1	

In the logic gate expressions, **A** and **B** are binary inputs (0 or 1), and **Z** is the resulting output. The following symbols are used to represent standard logic operations:

- $\cdot$ — AND operation
- $+$ — OR operation
- $\oplus$ — XOR (exclusive OR) operation
- $\overline{A}$ — NOT operation (negation of A)

These operators describe how inputs are combined to determine the gate's output.

3 Boolean Algebra Laws & Rules

Laws Involving 0 and 1	
Identity	$A + 0 = A$ $A \cdot 1 = A$
Null	$A + 1 = 1$ $A \cdot 0 = 0$
Idempotent	$A + A = A$ $A \cdot A = A$
Complement	$A + \overline{A} = 1$ $A \cdot \overline{A} = 0$
Involution	$\overline{\overline{A}} = A$

Algebraic Laws	
Commutative	$A + B = B + A$ $A \cdot B = B \cdot A$
Associative	$(A + B) + C = A + (B + C)$ $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$ $A + (B \cdot C) = (A + B)(A + C)$

De Morgan's Laws	
De Morgan	$\overline{A \cdot B} = \overline{A} + \overline{B}$ $\overline{A + B} = \overline{A} \cdot \overline{B}$

4 Applications in Real Life

- **Microchip, Processor, and Memory Design**: Essential for the development of integrated circuits, processors, and storage systems used in computers and electronic devices.
- **Complex Decision-Making Systems**: Utilized in algorithms for systems that make automated decisions, such as robotics, AI, and control systems.
- **Control Systems**: Applied in the design and operation of everyday systems, such as traffic lights, washing machines, and elevators, ensuring efficient and reliable functioning.
- **Information Retrieval**: Helps improve search algorithms, such as those used by search engines, to deliver precise and relevant results based on user queries.

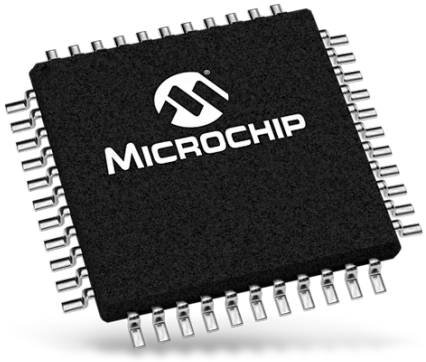


Figure 1: Microchip

Figure 2: Flash Memory

Lecture 2: April 23rd 2025

1 Half Adder

A half adder is a logic circuit that adds two binary digits, producing two outputs: Sum and Carry.

1.1 Truth Table

A	B	Z_C (Carry)	Z_S (Sum)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

1.2 Boolean Expressions

$$Z_S = \overline{A} \cdot B + A \cdot \overline{B} \quad \Rightarrow \quad Z_S = A \oplus B$$

$$Z_C = A \cdot B$$

The sum output Z_S is 1 when the inputs differ (i.e., $A = 0, B = 1$ or $A = 1, B = 0$), resulting in:

$$Z_S = \overline{A} \cdot B + A \cdot \overline{B} = A \oplus B$$

The carry output Z_C is 1 only when both inputs are 1:

$$Z_C = A \cdot B$$

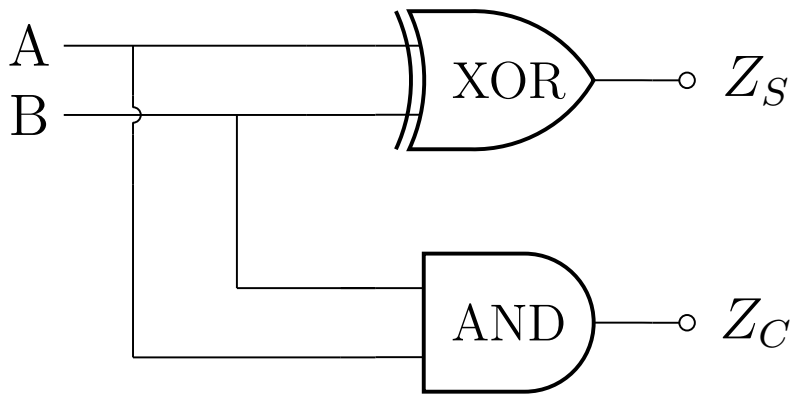


Figure 3: Half Adder Circuit Diagram

- **Inputs:** Two binary digits, A and B.
- **Outputs:**
  - **Sum (Z_S):** Calculated using XOR: $\boxed{Z_S = A \oplus B}$
  - **Carry (Z_C):** Calculated using AND: $\boxed{Z_C = A \cdot B}$

2 Full Adder

A full adder is a combinational logic circuit that adds three binary digits, including a carry-in, to produce a sum and carry-out. It is an extension of the half adder, which only adds two binary digits.

2.1 Truth Table

C (Carry-in)	A	B	Z_C (Carry-out)	Z_S (Sum)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

2.2 Canonical Boolean Expressions

The canonical Boolean expressions for the sum Z_S and carry-out Z_C are derived directly from the truth table:

$$Z_S = \overline{A} \cdot \overline{B} \cdot C + A \cdot \overline{B} \cdot \overline{C} + \overline{A} \cdot B \cdot \overline{C} + A \cdot B \cdot C$$

$$Z_C = A \cdot B \cdot \overline{C} + \overline{A} \cdot B \cdot C + A \cdot \overline{B} \cdot C + A \cdot B \cdot C$$

2.3 Minimized Boolean Expressions

For Z_S:

The minimized expression for the sum Z_S can be derived by grouping terms to form XOR operations:

$$\begin{aligned} Z_S &= (\overline{A} \cdot B + A \cdot \overline{B}) \cdot \overline{C} + (\overline{A} \cdot C + A \cdot \overline{C}) \cdot B \\ &= (A \oplus B) \cdot \overline{C} + (A \oplus B) \cdot C \\ &= (A \oplus B) \cdot (\overline{C} + C) \\ &= A \oplus B \oplus C \end{aligned}$$

Final Expression: $\boxed{Z_S = A \oplus B \oplus C}$

For Z_C:

The minimized expression for the carry-out Z_C is simplified using the identity $C + \overline{C} = 1$:

$$\begin{aligned} Z_C &= A \cdot B \cdot (C + \overline{C}) + (\overline{A} \cdot B + A \cdot \overline{B}) \cdot C \\ &= A \cdot B + (A \oplus B) \cdot C \end{aligned}$$

Final Expression: $\boxed{Z_C = A \cdot B + (A \oplus B) \cdot C}$

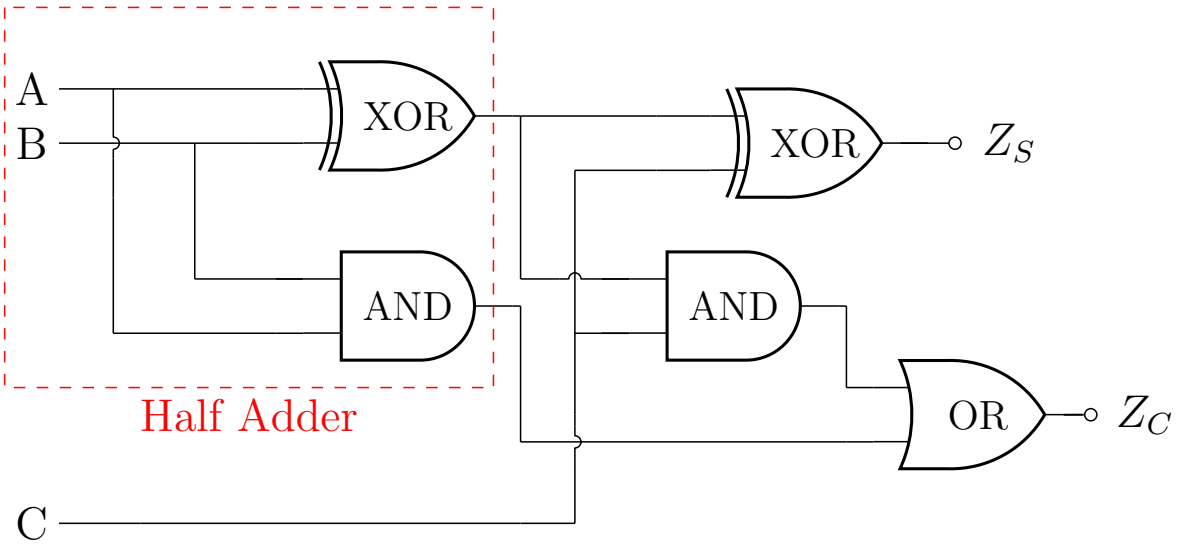


Figure 4: Full Adder Circuit Diagram

Lecture 3: April 29th 2025

1. POS, SOP, K-Maps, Excess-3 Code

1.1 Product of Sums (POS)

POS expresses a logic function as an AND of OR terms. It's derived by grouping the 0s from the truth table.

1.2 Sum of Products (SOP)

SOP expresses a logic function as an OR of AND terms. It's derived by grouping the 1s from the truth table.

1.3 K-Map Overview

- K-maps simplify Boolean expressions.
- Group 1s for SOP; group 0s for POS.
- Always group in powers of 2 (1, 2, 4, 8, ...).

1.4 SOP of an OR Gate

$$F = A + B$$

$$\begin{aligned} Q.Z &= \overline{A} + A\overline{B} + ABC' \\ &= \overline{A}(B + \overline{B})(C + \overline{C}) + A\overline{B}(C + \overline{C}) + ABC' \\ &= (\overline{A}B + \overline{AB})(C + \overline{C}) + A\overline{B}(C + \overline{C}) + ABC' \\ &= \overline{A}BC + \overline{A}B\overline{C} + \overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C} + A\overline{B}C + A\overline{B}\overline{C} + ABC' \end{aligned}$$

1.5 K-map Representations

K-map Table 1: Variable Indexing

	$\overline{B}$	B
$\overline{A}$	–	1_1
A	1_2	1_3

SOP of OR Gate

K-map Table 2: Variable Assignments

	$\overline{B}\,\overline{C}$	$\overline{B}C$	BC	$B\overline{C}$
$\overline{A}$	–	1_1	1_5	1_2
A	1_4	1_5	–	1_6

1.6 Truth Table and K-map Analysis

Truth Table:

A	B	C	Z
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

K-map (C vs AB):

	00	01	11	10
C = 0	1	1	1	1
C = 1	1	1	0	1

From the K-map:

- Group all 1s in rectangles of size 4.
- Identify literals that remain constant.

$$Z = \overline{B} + \overline{A} + \overline{C}$$

2. Excess-3 Code

- Excess-3 code is a self-complementary binary-coded decimal (BCD).
- To convert to Excess-3, add 3 to the decimal number and then convert to 4-bit binary.

2.1 Decimal to Binary and Excess-3

Decimal to Binary and Excess-3

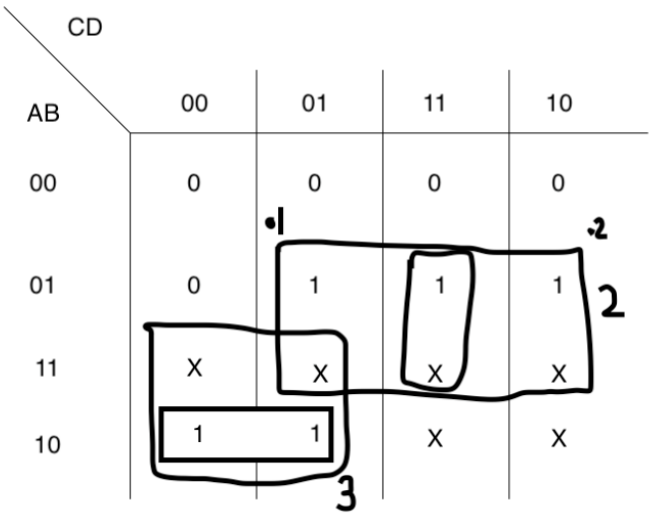
Decimal	Binary	Excess-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Note: For cells not defined by the truth table, mark them as `X`. These are called "don't care" conditions and can help simplify K-maps.

2.2 Self-Complementing Pairs

- (9, 0), (8, 1), (7, 2), (6, 3), (5, 4)

$$Z = A\overline{B}\,\overline{C} + \overline{A}BD + \overline{A}BC$$



2.3 4-variable K-map Representation

	$\overline{C}\,\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\,\overline{B}$	–	1_1	1_3	1_2
$\overline{A}B$	–	1_5	1_7	1_6
AB	1_{12}	1_{13}	1_{15}	1_{14}
A$\overline{B}$	1_8	1_9	–	–

$$Z = A\overline{C} + BD + BC$$

CREATING LOOPS IN K-MAPS

Before proceeding to the questions, ensure your conceptual clarity

Each cell of a Karnaugh map represents an output value for a given combination of inputs. Once all values (1's and 0's) are placed, all 1's must be grouped according to these core rules:

- Groups must consist of 2^n adjacent 1's (e.g., 1, 2, 4, 8, etc.).
- Make groups as large as possible while adhering to the power-of-two rule.
- Groups must be rectangular or square in shape.
- Larger groups simplify the function more, as each doubling eliminates one variable from the term.
- Every 1 must be included in at least one group.
- Overlapping is allowed; cells may appear in multiple groups.
- Redundant groups (fully contained within others) are unnecessary.
- Multiple valid minimal expressions may exist depending on grouping choices.

Q1 – Loop Identification and Simplification

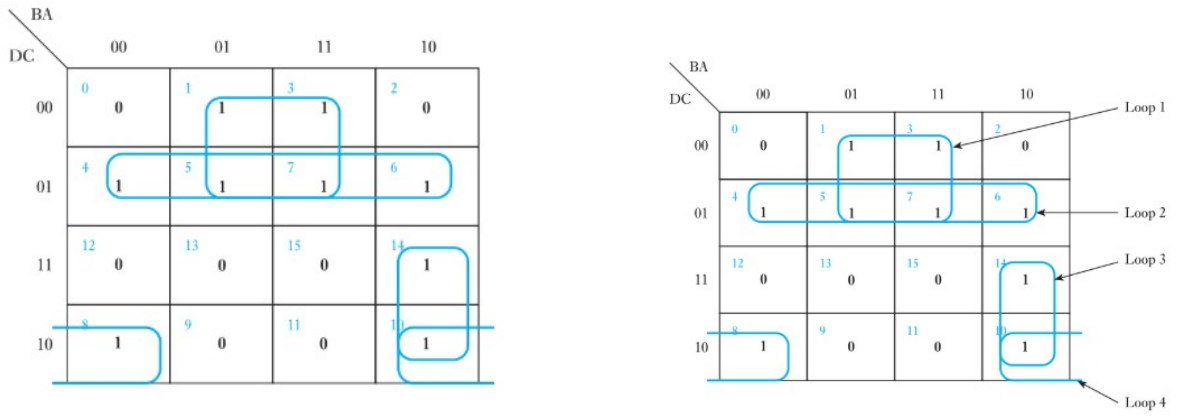
Problem Statement:
Consider the K-map shown in the figure from the previous question. Illustrate a valid set of loops and generate the corresponding simplified logic expressions for each loop.

Based on the groupings made in the Karnaugh map, the following expressions are derived for each loop:

Loop 1: $A \cdot \overline{D}$
Loop 2: $C \cdot \overline{D}$
Loop 3: $\overline{A} \cdot B \cdot D$
Loop 4: $\overline{A} \cdot \overline{C} \cdot D$

Combining all simplified terms, we get the final minimized Boolean expression:

$$G = A \cdot \overline{D} + C \cdot \overline{D} + \overline{A} \cdot B \cdot D + \overline{A} \cdot \overline{C} \cdot D$$



(a) Loop groupings in the K-map **(b)** Labelled K-map for expression generation

Figure 5: Illustration of loop groupings and derived expressions

LOGIC FUNCTIONS AND NETWORK, COMBINATIONAL LOGIC

Q1 – Logic Gate Design

An electrical control system utilizes three positional sensing devices, each generating a logic output of 1 when its respective position is confirmed. These devices are to be connected to a logic network composed of AND and OR gates. The system output should be logic 1 when **two or more** of the sensing devices are active (i.e., producing a logic 1).

Solution:

Let the sensor inputs be A, B, and C. The output is 1 when two or more inputs are 1, which corresponds to the minterms:

$$F = AB\overline{C} + A\overline{B}C + \overline{A}BC + ABC$$

Simplifying by factoring and combining terms:

$$\begin{aligned} F &= AB(\overline{C} + C) + AC(\overline{B} + B) + BC(A + A) \\ &= AB + AC + BC \end{aligned}$$

$$\boxed{F = AB + AC + BC}$$

This means the output is 1 whenever any two or more inputs are 1.

Logic Diagram: Uses three 2-input AND gates and one 3-input OR gate.

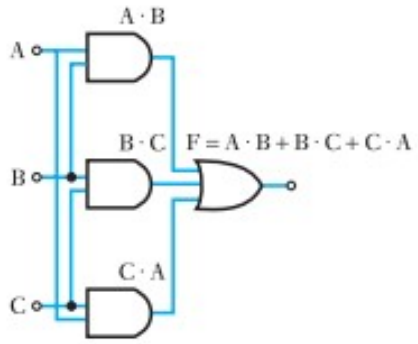


Figure 6: Logic gate implementation of $F = AB + AC + BC$

Q2 – Logic Gate Simplification and Circuit Design

Draw the circuit of gates that would effect the function:

$$F = \overline{A + B \cdot C}$$

Simplify this function and hence redraw the circuit that could implement it.

Solution

Original Expression:

$$F = \overline{A + B \cdot C}$$

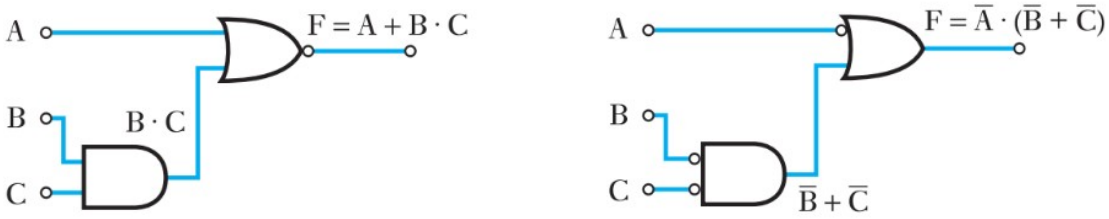
Apply De Morgan's Theorem:

$$\begin{aligned} F &= \overline{A} \cdot \overline{B \cdot C} \\ &= \overline{A} \cdot (\overline{B} + \overline{C}) \end{aligned}$$

$$\boxed{F = \overline{A} \cdot (\overline{B} + \overline{C})}$$

Implementation: Requires three NOT gates, one OR gate, and one AND gate.

Note: Simplification reduces complexity but may increase gate count due to added inverters.



(a) Original: $F = \overline{A + B \cdot C}$ **(b)** Simplified: $F = \overline{A} \cdot (\overline{B} + \overline{C})$

Figure 7: Original vs. Simplified Logic Implementation

Q3 – Implementing Logic Function Using NOR and NAND Gates

Draw circuits which will generate the function:

$$F = B \cdot (\overline{A} + \overline{C}) + \overline{A} \cdot \overline{B}$$

using:
(a) Only NOR gates
(b) Only NAND gates

Solution

Simplify the Expression:

$$\begin{aligned} F &= B(\overline{A} + \overline{C}) + \overline{A} \cdot \overline{B} \\ &= B \cdot \overline{A} + B \cdot \overline{C} + \overline{A} \cdot \overline{B} \\ &= B \cdot \overline{C} + \overline{A} \cdot (B + \overline{B}) \\ &= B \cdot \overline{C} + \overline{A} \cdot 1 \\ &= \boxed{F = \overline{A} + B \cdot \overline{C}} \end{aligned}$$

(a) NOR Gate Implementation

Use double negation and De Morgan's Theorem:

$$\begin{aligned} F &= \overline{\overline{\overline{A} + B \cdot \overline{C}}} \\ &= \overline{\overline{\overline{A} \cdot \overline{(B \cdot \overline{C})}}} \end{aligned}$$

This form uses only NOR gates.

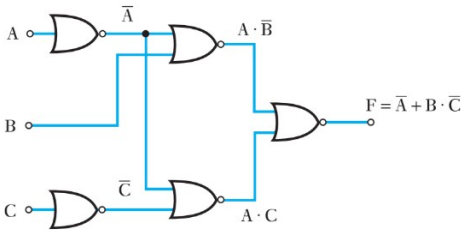


Figure 8: Q3(a) – NOR Gate Implementation

(b) NAND Gate Implementation

Use the same simplified form:

$$F = \overline{A} + B \cdot \overline{C}$$

Rewritten using NAND logic:

$$F = \overline{\overline{\overline{A} \cdot \overline{(B \cdot \overline{C})}}}$$

Use:

- NAND with identical inputs for inverters $(\overline{A}, \overline{C})$,
- NAND for AND and OR equivalents using De Morgan identities.

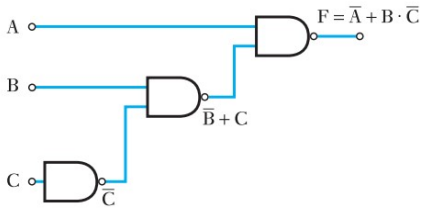


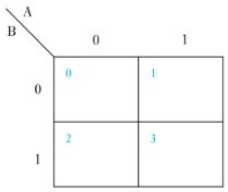
Figure 9: Q3(b) – NAND Gate Implementation

KARNAUGH MAPS (K-MAPS) FOR SIMPLIFYING COMBINATIONAL LOGIC

Q1 – Karnaugh Map for Two Variables

Problem Statement: Fill out the Karnaugh map given $G = 1, 2, 3$ with input variables B and A.

We are given that G depends on input variables A and B, indicating a two-variable Karnaugh map, shown in Fig. 4(a). The binary cell numbers correspond to the input combinations of B (row) and A (column).



B	A	Output	Binary code
0	0		$00_2 = 0_{10}$
0	1		$01_2 = 1_{10}$
1	0		$10_2 = 2_{10}$
1	1		$11_2 = 3_{10}$

Figure 10: Q1(a): Karnaugh map with cell numbering for two variables

The expression $G = 1, 2, 3$ means the output is 1 for minterms 1, 2, and 3. These correspond to:

$$01 \Rightarrow A = 1,\ B = 0$$
$$10 \Rightarrow A = 0,\ B = 1$$
$$11 \Rightarrow A = 1,\ B = 1$$

Filling the map accordingly yields Fig. 4(b):

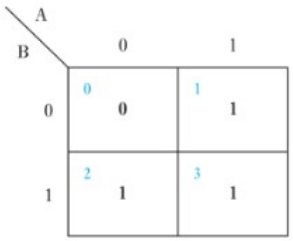


Figure 11: Q1(b): Filled K-map for $G = 1, 2, 3$

Note: The resulting function is $\boxed{G = A + B}$, an OR gate.

Q2 – Four-Variable Karnaugh Map

Problem Statement: Fill out the Karnaugh map given $F = 0, 5, 6, 8, 12, 13$ with input variables D, C, B, A.

The output F depends on inputs A, B, C, D, requiring a four-variable Karnaugh map as shown in Fig. 5(a). Each cell corresponds to a 4-bit binary input combination, with binary indexing based on D, C, B, A.

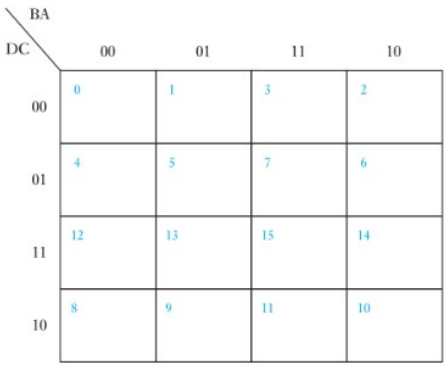


Figure 12: Q2(a): Karnaugh map layout for four variables

The minterms $F = 0, 5, 6, 8, 12, 13$ represent the following input combinations:

$$0000 \Rightarrow \text{minterm } 0$$
$$0101 \Rightarrow \text{minterm } 5$$
$$0110 \Rightarrow \text{minterm } 6$$
$$1000 \Rightarrow \text{minterm } 8$$
$$1100 \Rightarrow \text{minterm } 12$$
$$1101 \Rightarrow \text{minterm } 13$$

These cells are marked with 1s; all others are 0. The filled K-map is shown in Fig. 5(b).

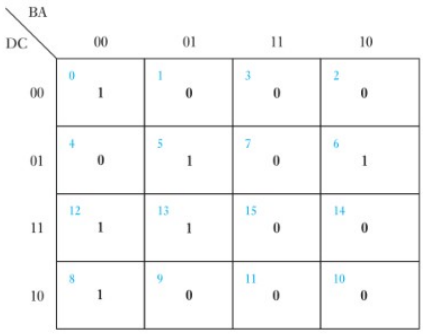


Figure 13: Q2(b): Completed Karnaugh map for given minterms

Note: Correctly translating decimal minterm indices into binary form is key to accurately filling the K-map.

Q3 – Karnaugh Map from Logic Expression

Problem Statement: Given the following logic expression, create and complete the corresponding Karnaugh map.

$$G = A \cdot \overline{B} \cdot \overline{C} \cdot \overline{D} + A \cdot B \cdot \overline{C} \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot C \cdot \overline{D} + A \cdot \overline{B} \cdot C \cdot \overline{D}$$
$$+ A \cdot B \cdot C \cdot \overline{D} + \overline{A} \cdot B \cdot C \cdot \overline{D} + \overline{A} \cdot B \cdot C \cdot D$$
$$+ \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot D + \overline{A} \cdot B \cdot \overline{C} \cdot D$$

Figure 14: Q3(a): Logic expression diagram

The output G is 1 for nine specific input combinations of variables A, B, C, D, each corresponding to a minterm. For example:

- $A \cdot b \cdot c \cdot d \Rightarrow A = 1,\ B = 0,\ C = 0,\ D = 0$
- $A \cdot B \cdot c \cdot d \Rightarrow A = 1,\ B = 1,\ C = 0,\ D = 0$
- $a \cdot b \cdot C \cdot d \Rightarrow A = 0,\ B = 0,\ C = 1,\ D = 0$
- etc.

All other input combinations yield an output of 0. These minterms are marked with 1s in the K-map; all other cells are 0.

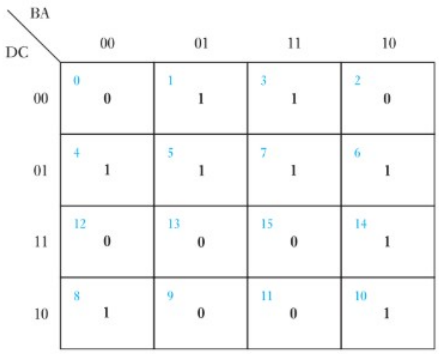


Figure 15: Q3(b): Completed Karnaugh map for the given expression

Note: Carefully match each product term to its binary equivalent when filling out a K-map.

Contribution Statement

- **Sumedh Nitin Jamsandekar**: Authored Lecture 1 notes and managed overall LaTeX compilation and document editing.
- **Aaron Binoj Amacadu**: Assisted in drafting and refining the content for Lecture 1.
- **Naman Rindani**: Compiled and structured the notes for Lecture 2.
- **Abhimanyu Prakash**: Wrote theoretical explanations and key concepts from Lecture 3.
- **Sarvesh Saravanan**: Formulated and answered conceptual questions based on Lecture 3.